

Solution Format Specification — MPVRP-CC

MPVRP-CC Team — benchmark specification

Note: This document details the file format used for MPVRP-CC solutions. To be validated, a solution must strictly follow the structure described below.

1. File Format

Solutions are stored in text files with the `.dat` extension. For official instance `MPVRP_001_s48_d1_p1.dat`, the canonical solution name is `Sol_MPVRP_001_s48_d1_p1.dat`. The submission service also accepts the short name `Sol_001.dat`. A ZIP may contain files at any directory depth, but it must contain one solution for every ID from `001` through `150`.

2. File Structure

The file describes the routes vehicle by vehicle. For each vehicle used, the solution contains a block of **2 lines**, separated by an empty line.

2.1 Line 1: Visit Sequence

```
ID: Garage - Depot [Load] - Station (Deliver) - ... - Garage
```

This line starts with the vehicle ID and describes the path:

- **Garage:** Start and end point (Node ID only).
- **Depot:** Identified by square brackets `[Qty]` indicating quantity loaded.
- **Station:** Identified by parentheses `(Qty)` indicating quantity delivered.

Node IDs refer to their 1-based index in the instance file (e.g., loaded at Depot 1, delivered to Station 2) and are not cumulative across types.

2.2 Line 2: Product Sequence and Costs

```
ID: Prod(Cost) - Prod(Cost) - ...
```

This line indicates which product is associated with every route step and the cumulative changeover cost. Products are zero-based in solutions: valid IDs are `0, ..., NbProducts - 1`.

The first token, at the departure garage, must be the vehicle's initial product from the instance converted to zero-based indexing. A product may change only on a depot step. The cumulative cost increases by the directed matrix value whenever the depot product differs from the preceding configuration, including before the first trip.

Important: The two lines must be perfectly aligned in terms of the number of steps. Each element in the visit sequence corresponds to exactly one element in the product sequence.

3. Valid Solution Example

```
1: 1 - 1 [1344] - 2 (1344) - 1
1: 0(0.0) - 0(0.0) - 0(0.0) - 0(0.0)

2: 1 - 1 [8947] - 1 (4278) - 2 (2350) - 3 (2319) - 1
2: 1(0.0) - 1(0.0) - 1(0.0) - 1(0.0) - 1(0.0)
```

In this example:

- **Vehicle 1** starts at garage 1, loads 1344 units at depot 1, delivers 1344 units to station 2, and returns to garage 1. It carries product 0 (cost 0.0).
- **Vehicle 2** starts at garage 1, loads 8947 units at depot 1, delivers to stations 1, 2, and 3, and returns to garage 1. It carries product 1 (cost 0.0).

4. Solution Metrics

After all vehicle routes, the file ends with **6 lines** of performance metrics, in the following order:

```
2
7
55.66
1385.07
Intel Core i7-10700K
0.245
```

4.1 Line 1 — Number of Vehicles Used

The count of vehicles with at least one delivery (e.g., **2**).

4.2 Line 2 — Number of Product Changes

The total number of product changes across the entire solution (e.g., **7**).

4.3 Line 3 — Total Transition Cost

The sum of all product changeover costs for all vehicles (e.g., **55.66**).

4.4 Line 4 — Total Distance

The total distance traveled by the fleet, expressed as the sum of Euclidean distances (e.g., **1385.07**).

4.5 Line 5 — Processor

The model of the processor on which the solution was generated (e.g., **Intel Core i7-10700K**).

4.6 Line 6 — Resolution Time

The time elapsed to generate the solution, in seconds (e.g., `0.245`).

A valid solution must satisfy all the constraints.

5. Feasibility rules enforced by the platform

- Every route block uses a distinct vehicle from the instance.
- A route starts and ends at that vehicle's home garage; garages cannot occur in the middle.
- Every mini-route starts with one positive depot load, contains at least one station delivery, and ends at a depot or the final garage.
- The product remains constant throughout a mini-route and may change only at a depot.
- A mini-route's loaded quantity equals its delivered quantity and never exceeds vehicle capacity.
- Quantities are positive, depot stocks are respected, and every station-product demand is met exactly.
- One vehicle may serve a station-product pair at most once across its complete route.
- Cumulative changeover costs and the six final metrics must agree with values recomputed by the verifier.

6. Ex-post changeover evaluation

A route produced from a zero-changeover instance can be evaluated afterward against the paired original matrix. The `mpvrp-reevaluate-changeovers` command preserves the route, products, loads, deliveries, distance, processor, and solve time. It changes only the cumulative values shown on product lines and the final number and total cost of product changes. By default, the source file is never overwritten; a repriced copy is created in a dedicated subdirectory.